11/03/26

→ Algoritmos → nem sempre é necessário melhorar um algoritmo, mas quando sempre é possível

↳ análise de algoritmos nasceu da necessidade de escolher algoritmos

↳ mais dados, mais esforço computacional

O que é

↳ São procedimentos com estrutura bem definida

↳ Não há meio termo

↳ Recebem um conjunto de valores (input)

↳ Produzem dados de saída (ou não)

↳ Ocupam espaço

↳ descrição do problema em etapas estruturadas e lógicas que mapeiam o input para o output

↳ Vetor é armazenado sequencialmente no memória

→ Instância de um problema
↳ uma representação concreta de um problema
↳ específico (entrada / saída)

↳ em algoritmos procuramos saber: execução, espaço, corretude

↳ algoritmos de ordenação são a melhor benchmark para análise de algoritmos

↳ O nível caso seria assembly, pois é mais transparente, mas não é fácil

↳ Performance em si da máquina não importa para a análise

↳ A máquina mais rápida pode dar execuções mais rápido, mas da máquina qual vai depender da eficiência do algoritmo

→ para saber a quantidade de segundos para execução:

$\frac{\text{total inst. algoritmo}(O)}{\text{inst./s da máquina}} \approx$ segundos

- **Exemplo:**
  - **Algoritmo 1:**
    - $\frac{2.(10^6)^2 \text{ instruções}}{10^9 \text{ instruções/segundo}} \approx 2000$ segundos
  - **Algoritmo 2:**
    - $\frac{50.(10^6 \log 10^6) \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 100$ segundos
  - Computador B é 20x mais rápido que o computador A.

$O(n) = 2n^2$ →

$O(n) = 50 \cdot n \log n$ →

$n = 10^6$

? número de elementos no vetor

→ algoritmo é tão importante quanto o hardware

→ devemos ter uma medida de eficiência que consiga estimar a eficiência do algoritmo

→ queremos mais poder computacional sem respeito à necessidade

→ quanto mais eficiente um algoritmo melhor

→ programador, linguagem, máquina podem afetar a execução do algoritmo. Contudo, um critério mais uniforme seria: tempo de execução, número de comparações, espaço.

→ podemos analisar tanto um algoritmo como uma classe de algoritmos

→ Análise de um único algoritmo
↳ análise de custo de um problema específico
↳ pode ser analisado: número de execução, quantidade de memória
↳ explicar partes do algoritmo depois uma todo

→ Análise de classe de algoritmos
↳ família de algoritmos
↳ identificar o que é melhor
↳ podemos definir um critério, como: estimar min/max de comparações

↳ fica mais fácil a gral

→ Custo de um algoritmo
↳ identificar o menor custo
↳ ≤ menor custo possível = algoritmo ótimo
↳ pode ser aplicado à uma classe toda, o que facilita a análise
↳ exige uma medida de dificuldade (comparação por exemplo)

→ Algoritmos estatísticos não buscam o espaço todo, podendo ser mais performático, contudo podem não dar a melhor resposta, ao contrário de um algoritmo determinístico

→ Algoritmo ótimo é passível de modificação, ou seja, pode melhorar no futuro.

→ Em uma classe de algoritmos, todos retornam a mesma coisa mas com custos diferentes

→ É necessário formar de analisar um teste algoritmo para cada instância

→ podemos analisar pelo tempo de execução em um modelo matemático

→ análise de custo pelo tempo não pode ser generalizada, pois depende de tudo: compilador, Hardware, OS, etc.

→ Para uma análise mais sólida
↳ necessário um modelo matemático
↳ uma um computador ideal
↳ medida mais fiel da tendência do algoritmo
↳ mas medida mais significativa uma ainda ao hardware
↳ Uma de exemplo número de comparações (que aumentam custo) e operações aritmética, atribuição, lógica, etc.

↳ função de complexidade → do input a saída
↳ pode ser em espaço ou tempo

Função de Complexidade de Tempo: Exemplo

- Algoritmo para encontrar o maior elemento em um vetor de inteiros $A[0..n\text{-}1]$, $n \geq 1$.

Programa 1

```
int Max (int *A, int n)
{  int i, max;
   max=A[0];
   for (i=1;i<n;i++)
   {
     if (max<A[i])
        max=A[i];
   }
   return max;
}
```

- Seja f uma função de complexidade tal que:
  - $f(n)$ ⇒ **número de comparações** entre os elementos de A, se A contiver n elementos.
  - Logo, $f(n) = n\text{-}1$, para $n > 0$.
  - Provar que o algoritmo apresentado no *programa 1* é **ótimo**.

comparação é o que mais pesa. Como é uma busca sequencial, o custo é $f(n) = n-1$ quando $n > 0$

Função de Complexidade de Tempo: Exemplo

- **Teorema:**
  - **Qualquer algoritmo** para encontrar o maior elemento de um conjunto com n elementos (não ordenados), $n > 1$, **faz pelo menos** n -1 **comparações**
- **Prova:**
  - Cada par de elementos **tem de ser comparado**.
  - O resultado é um número selecionado
  - Processo iniciado a partir do 2º. Elemento.
  - Logo, n-1 comparações são necessárias.

Se o número de comparações for utilizado como medida de custo: **algoritmo é ótimo**.

Prova que é ótimo

Custo uniforme para qualquer n!

→ o que implica é o tamanho da entrada

→ melhor caso → menor tempo
→ pior caso → pior tempo
→ caso médio → médio do tempo
↳ caso esperado
↳ distribuição das probs. sobre o conjunto de entradas de tamanho n
↳ nem sempre é o que acontece

Exemplo 1: Pesquisa Sequencial

Resolução

- Seja f uma função de complexidade ⇨ $f(n)$ é o **número de registros pesquisados**
  - **melhor caso**: $f(n) = 1$ ⇨ registro procurado é o **primeiro** consultado;
  - **pior caso**: $f(n) = n$ ⇨ registro procurado é o **último** consultado ou **não está presente** no arquivo;

Exemplo 1: Pesquisa Sequencial

Resolução

- caso médio

o que a vez pode não ser verdad

- Considerar que toda pesquisa recupera um registro.
- Se p_i ➡ probabilidade ➡ encontrar o i-ésimo registro ➡ i comparações, então
- $f(n) = 1 \,.\, p_1 + 2 \,.\, p_2 + 3 \,.\, p_3 + \ldots + n \,.\, p_n.$

numero da iteração

Exemplo 1: Pesquisa Sequencial

- Para calcular $f(n)$
  - Considerar uma distribuição de probabilidades p_i
    - **Igualmente Provável** para acessar todos os registros, então:

$$p_i = \frac{1}{n}, 1 \le i \le n.$$

← todos tem a mesma prob de ser acessado

  - Neste caso:

$$f(n) = 1.\frac{1}{n} + 2.\frac{1}{n} + 3.\frac{1}{n} + \ldots n.\frac{1}{n}$$

$$f(n) = \frac{1}{n}.(1 + 2 + 3 + \ldots + n)$$

→ PA

$$f(n) = \frac{1}{n}.\left(\frac{n.(n+1)}{2}\right) \quad \text{(PA)}$$

$$f(n) = \frac{(n+1)}{2}$$

← comportamento linear

→ comportamento linear é o dobro, mas difícil de se descobrir